A Circuit Verification Tool

In this project you write a small **circuit verification tool**. It reads a circuit and a specification, and decides whether the circuit satisfies the specification for *all* inputs, e.g. whether a multiplier really multiplies, or whether two outputs are never both 1. As in the lecture, this is an **ideal membership** problem, and your tool decides it with your own implementation of **Buchberger's algorithm** in **SageMath**. If the specification does not hold, your tool reports a nonzero remainder, which helps to find the inputs that violate it.

1 Circuit Verification as Ideal Membership

Every logical gate of a circuit raises a polynomial equation in $\mathbb{Q}[X]$, e.g.

$$h = f \wedge g \;\rightsquigarrow\; -h + fg, \qquad h = f \vee g \;\rightsquigarrow\; -h + f + g - fg,$$
$$h = f \oplus g \;\rightsquigarrow\; -h + f + g - 2fg, \qquad h = \neg f \;\rightsquigarrow\; -h + 1 - f,$$

and every input x gives the Boolean constraint $x^2 - x$. Let F be the gate polynomials and the Boolean constraints of a circuit. The circuit satisfies its specification $S = 0$ for all inputs if and only if $S \in \langle F \rangle$.

A specification can be a relation between the outputs and the inputs, e.g. $S = 8s_3 + 4s_2 + 2s_1 + s_0 - (2a_1 + a_0)(2b_1 + b_0)$ for a 2-bit multiplier, but also another property of the circuit, e.g. $S = s_0 s_1$ for a half adder: its sum and carry bits are never both 1.

Your tool must work for every order of the variables. In some orders the input is already a Gröbner basis. In other orders it is not, and your Buchberger algorithm has real work to do.

2 Setting up SageMath

You need a SageMath installation that can run Python scripts. Any of the following works:

- **SageMath** (version 10 or newer), see `https://doc.sagemath.org/html/en/installation/`, e.g. via conda: `conda create -n sage -c conda-forge sage`. Run scripts with `sage -python <script.py> ...`.
- **passagemath**, a pip-installable distribution of SageMath. A small installation suffices for this project:

```
python3 -m venv sage-venv
sage-venv/bin/pip install passagemath-singular
sage-venv/bin/python <script.py>
```

The template `project_template.py` works with both (see the `import` at its top).

3 The Task

Your tool consists of the four functions of `project_template.py`. Rename the file to `project_<surname>_<matrnr>.py` and implement them as specified in the template:

- `divide(f, F)`: the division algorithm; it returns only the remainder, not the quotients,
- `spoly(f, g)`: the S-polynomial,
- `buchberger(F)`: Buchberger's algorithm, with the shortcut for pairs whose leading monomials are relatively prime; it also counts the pairs (see the template),
- `member(f, F)`: whether $f \in \langle F \rangle$, decided with your functions above.

The given code reads the circuit and calls `member(S, F)`. Do not change the names, the arguments, or the form of the results of the four functions, and do not change the given code for input and output. Your functions must work for all inputs as specified, not only for the test examples.

Allowed: SageMath's polynomial arithmetic and its basic functions on polynomials and monomials (leading term, leading monomial, leading coefficient, ...). Have a look at `https://doc.sagemath.org/html/en/reference/polynomial_rings/polynomial_rings_multivar.html`

Not allowed:

- ideals: `R.ideal(...)`, `ideal(...)`, and all methods of ideals, in particular `groebner_basis()`, `reduce()`, `interreduced_basis()`, `basis_is_groebner()`, `normal_basis()`, `transformed_basis()`, `elimination_ideal()`, `variety()`, `syzygy_module()`, and the membership test `f in I`,
- division and reduction of polynomials: `f.reduce(...)`, `f.quo_rem(g)`, `f.lift(...)`, `f.mod(...)`, `f % g`, `f // g`,
- other computer algebra systems called from SageMath, e.g. `singular` and `macaulay2`.

Using one of them in a function gives 0 points for this function. You may use them to test your code (e.g. to compare your results with `groebner_basis()`), but your submitted file must not contain them.

4 Input, Output, and Test Examples

A circuit is given by the variables (in decreasing order), the monomial order (`lex`, `deglex` = grlex, or `degrevlex` = grevlex), a line `spec` with the specification polynomial S, and the gate polynomials and Boolean constraints, one per line; lines starting with # are comments. The given code prints whether the circuit is correct, and the remainder of S on division by a Gröbner basis:

```
sage -python project_<surname>_<matrnr>.py tests/multiplier_2bit.ideal
```

The 2-bit multiplier with the input on the left and the output on the right:

```
# 2-bit multiplier                                          s CORRECT
vars s3 s2 s1 s0 c1 p11 p10 p01 p00 a1 a0 b1 b0             r 0
order lex
spec 8*s3 + 4*s2 + 2*s1 + s0 - (2*a1 + a0)*(2*b1 + b0)
-s3 + p11*c1
-s2 + p11 + c1 - 2*p11*c1
-c1 + p01*p10
-s1 + p01 + p10 - 2*p01*p10
-s0 + p00
-p00 + a0*b0
-p01 + a0*b1
-p10 + a1*b0
-p11 + a1*b1
a1^2 - a1
a0^2 - a0
b1^2 - b1
b0^2 - b0
```

With an OR gate for s_1 instead (`tests/multiplier_2bit_bug.ideal`), the output is

```
s INCORRECT
r 2*a1*a0*b1*b0
```

The folder `tests/` contains test circuits (`*.ideal`): small adders, multipliers, and other circuits, correct and buggy, with the expected output (`*.out`). Your tool should need at most a few seconds for each test circuit.

5 Submission

Submit your solution `project_<surname>_<matrnr>.py` (a single Python file) to TUWEL before 20. November 2026 23:59.